

Youth Basketball League Management System: A Relational Database Design Proposal

Khan Wasi, Arman Madatyan, Bryant Tran

CPSC 332 #18350

California State University, Fullerton

Professor Ken Wuttisasiwat

03-26-2026

System Description

The Youth Basketball League Management System is the proposed relational database model that is designed to help a youth basketball organization function for its full lifecycle. The Youth Basketball League Management System aims to help with all day to day operational data into a relational schema where coaches, guardians, referees, and league administrators are able to reliably use. The primary users for this database are the league administrators who need to configure and oversee the entire system every season and coaches; whom of which, are assigned to one or multiple teams. These coaches require access to roster information which contain players. Referees are assigned to games based on the availability and certification level or credibility. The database will track players' basic demographics such as age, team composition, coaching assignments, and general game scheduling across physical locations as well as registration and payment transaction histories.

Scope

Below are the scope of our Youth Basketball League Management System and the following things it will track or function.

- The system will track player demographics, eligibility status, and guardian contact information for every registered participant.
- The system will manage team rosters, including player assignments that are scoped to a specific season to preserve historical records.

- The system will record and enforce multi-season coaching assignments, supporting scenarios where a single coach manages more than one team.
- The system will schedule games between teams, capturing gym location, date, start time, and final score outcomes.
- The system will assign one or more certified referees to each scheduled game and maintain a log of referee participation history.
- The system will track gym availability and prevent the double-booking of a facility at the same date and time.
- The system will manage guardian-initiated player registrations, including status tracking (Active, Waitlisted, or Withdrawn) throughout the season.
- The system will record payment transactions associated with each registration, supporting statuses of Pending, Completed, and Refunded.
- The system will support multiple seasons with full historical data retention, enabling year-over-year reporting on teams, players, and league performance.
- The system will enforce referential integrity across all related entities, ensuring that no orphaned records exist for payments, registrations, or game assignments.

UML Overview (High-Level Entity-Relationship Model)

The following section provides a high-level description of the entities, attributes, and relationships that constitute the YBLMS relational schema.

Entities and Representative Attributes

One way the YBLMS database works is by tracking details about players, teams, matches, and how the league runs. What holds it together are main data groups linked through helper tables that connect entries precisely. Sometimes connections matter just as much as the records themselves. Through structured setups, consistency stays intact across updates. Each piece fits because cross-reference points keep things aligned. Entries relate cleanly when support structures handle overlaps. Accuracy improves when links reflect real-world ties.

A single player gets tracked through their own ID, name, birth date - also tied to someone responsible. That person shows up in the system by name, reachable via email or call, every address different. One grown-up might watch over several kids on file. Connections form between younger members and the adult listed, building links across records behind the scenes. Each squad gets listed by its own number, what it is called, which group it plays in - also tied to when that round of games happens. A time frame for play appears through labels showing the title, first day, last day - shaping how long things run.

Information about each coach lives inside the Coach container - names show up here, alongside contact details like emails and telephone numbers. Inside the Game structure, every match gets its own spot, linking to both participating teams plus the venue hosting it, while holding onto

when things kick off. A place to keep physical site facts appears in the Gym unit: think names of buildings, street locations, towns they sit in, how many people can fit inside.

A referee's details live inside the Referee entity - certification grade sits alongside phone numbers and email addresses. Because of this setup, game assignments happen smoothly behind the scenes.

Junction and Lookup Tables

Junction tables handle many-to-many links across the system. Linking athletes to squads per season happens through the Registration table. A distinct registration ID marks each entry there. Player, team, and season details tie in via external keys. The date of sign-up appears alongside a status code. Tracking involvement precisely rests on these records.

A single game might have more than one official watching over it. That link between matches and those officials lives inside the GameReferee table. Instead of a single unique number, this table combines both the match code and the referee identifier as its main key. This setup makes room for several referees to tie into just one event. The connection runs both ways - neither side gets left out.

It keeps things uniform by using reference charts for status codes. PaymentState holds options including pending, finished, canceled. Another one called SigninStatus tracks labels such as live, queued, pulled out. Consistency through the data improves because of these fixed categories.

Relationships and Cardinality

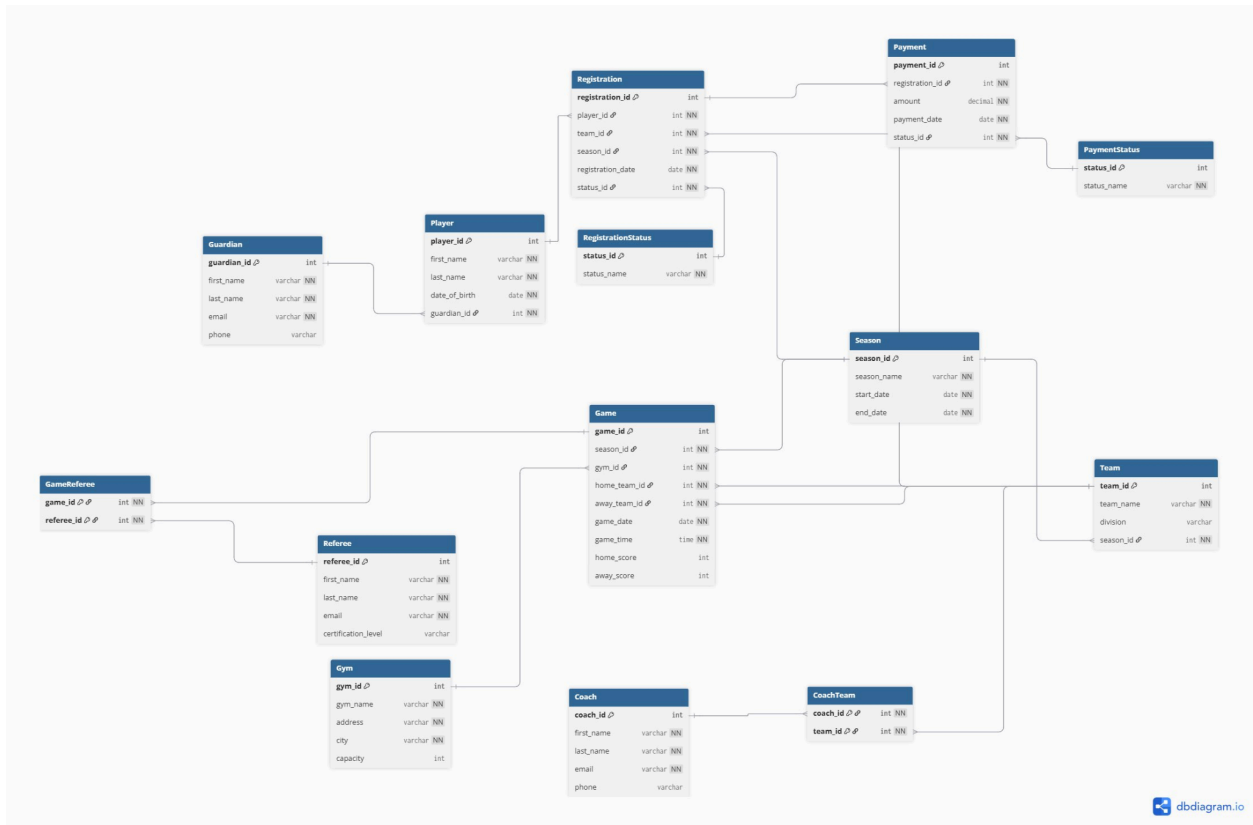
One guardian might link to more than just a single player. Teams show up across several seasons, each holding their own place within one season's structure. Games pile up under a single team's name over time. Multiple matchups connect back to that same team again and again.

A single gym holds every match, yet each court might see several matchups over time - showing how locations link to multiple events. One season ties together all those games, no matter where they happen.

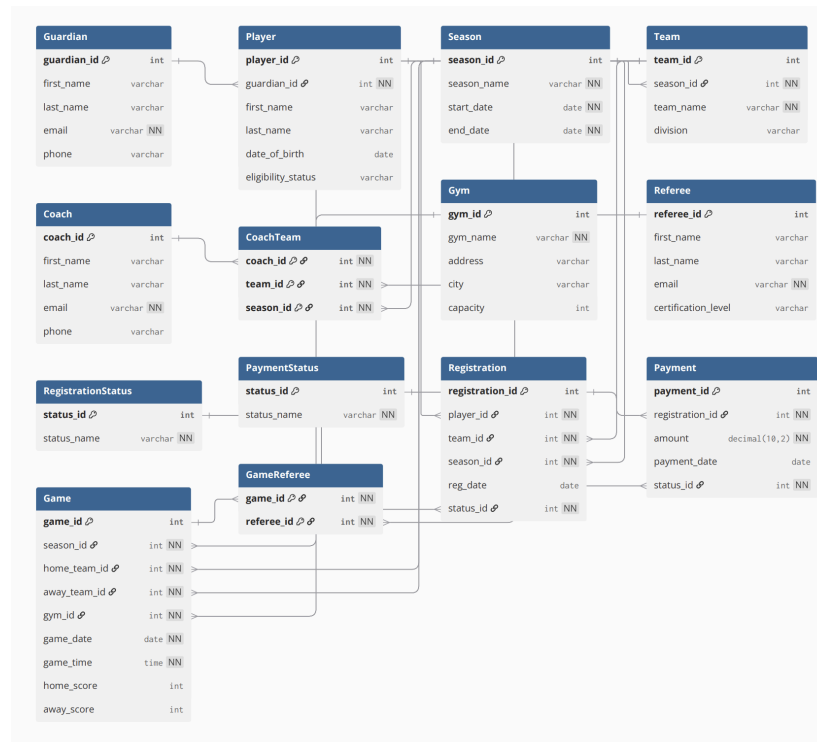
A single player might link to several teams, just as one team hosts numerous players - this tie needs a bridge, built here by the registration table. Seasons shift, yet records stay clear because every entry pins a player to a team at a certain time. When it comes to games and referees, connections multiply fast - one match often involves more than one official. The GameReferee table keeps those links precise, slotting each referee into the right game context.

Few realize how fluid it gets - one coach guiding several squads at once while each squad pulls advice from more than one mentor.

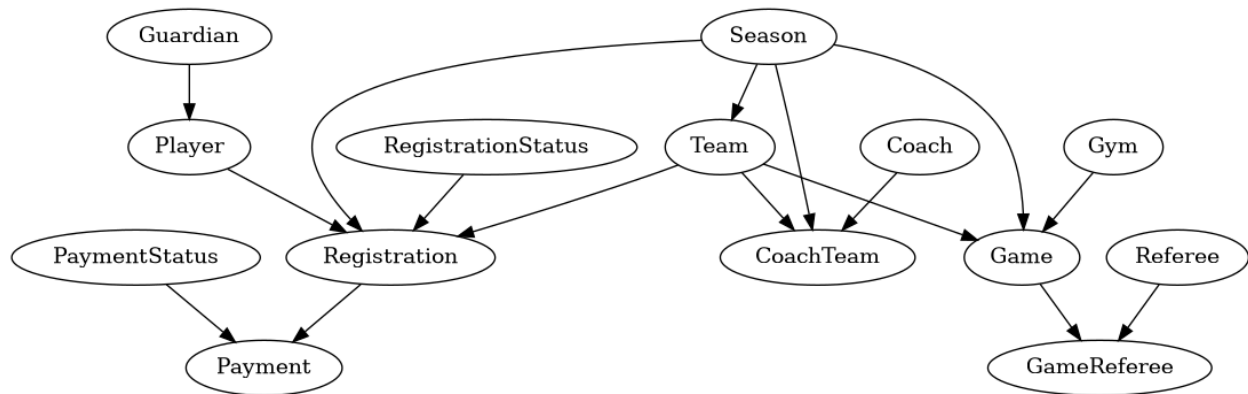
The system includes a minimum of two M:N relationships. The Registration table serves as the junction between Player, Team, and Season — capturing the fact that a player's team membership is seasonal and historically preserved. The GameReferee table resolves the M:N relationship between Game and Referee, allowing each game to have multiple officiating staff while tracking each referee's assignment history across all games in a season.



dbdiagram.io



OVERVIEW



Business Rules and Constraints

Here are the following rules that the business must follow and the constraints that are placed onto the database itself. These rules will be implemented through primary keys, foreign keys, unique constraints, check constraints, and application-layer validation logic.

01: Guardian Email Uniqueness

The email field in the Guardian table must be unique. No two guardians may share the same email address, as email serves as the primary login and communication identifier.

02: Coach Email Uniqueness

The email field in the Coach table must be unique across all coach records, ensuring unambiguous identification of each coaching staff member.

03: Seasonal Team Membership

A player may be registered to only one team per season. The Registration table enforces a composite unique constraint on (player_id, season_id) to prevent duplicate team assignments within the same seasonal period.

04: Season Date Validity

A season's end_date must be strictly greater than its start_date. This constraint prevents the creation of invalid or zero-length seasons.

05: Game Date Within Season

A game's game_date must fall on or between the start_date and end_date of its associated season. Games cannot be scheduled outside of the active seasonal window.

06: No Gym Double-Booking

No two games may be scheduled at the same gym_id with the same game_date and game_time. A composite unique constraint on (gym_id, game_date, game_time) in the Game table enforces this restriction.

07: Self-Play Prevention

A game's home_team_id and away_team_id must reference two distinct teams. A team cannot be scheduled to play against itself.

08: Payment Referential Integrity

A payment record cannot exist without a corresponding valid registration. The registration_id in the Payment table is a foreign key that references Registration.registration_id.

09: Payment Status Constraint

The payment_status field must be one of the predefined values in the PaymentStatus lookup table: Pending, Completed, or Refunded. Direct string insertion outside of these values is not permitted.

10: Registration Status Constraint

The registration_status field must reference a valid value in the RegistrationStatus lookup table: Active, Waitlisted, or Withdrawn.

11: Referee Game Assignment

Every scheduled game must have at least one referee assignment recorded in the GameReferee table before the game's status can be set to Confirmed. This rule is enforced at the application layer during the scheduling workflow.

References

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.).

Addison-Wesley.

Wuttisasiwat, K. (2026). *CPSC 332: Database — lecture slides*. Canvas. California State

University, Fullerton.

dbdiagram.io. (2024). *Free database diagram (ERD) tool*. <https://dbdiagram.io>